

ใบงาน Week 6-8

MiniShop React: From UI to Interactive Web Application

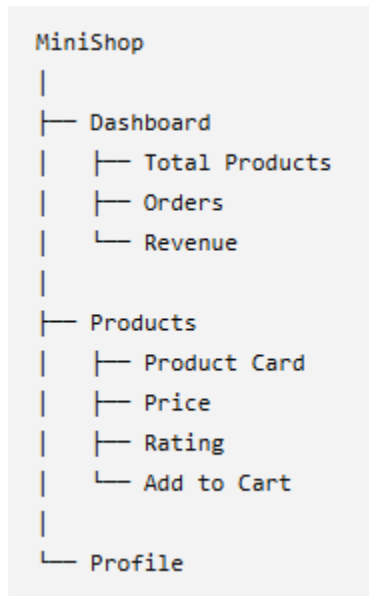
Project: MiniShop React Application

เครื่องมือ: VS Code + Node.js + npm + Vite + React + Tailwind CS

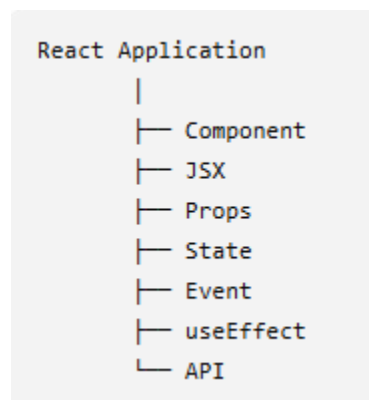
1. ภาพรวมของใบงาน

ใน Week 3 นักศึกษาได้สร้างหน้าร้าน **MiniShop** โดยใช้

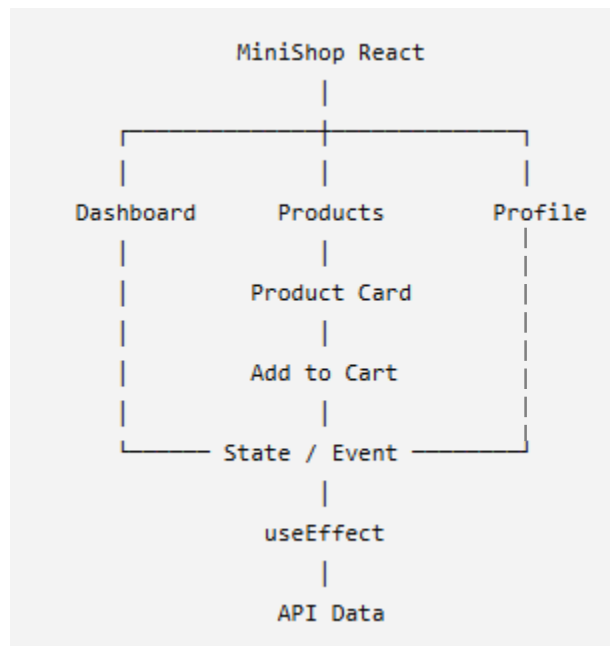
HTML, CSS / Tailwind CSS, JavaScript ตัวอย่างเช่น



ใน Week 6 - 8 จะเป็นการนำแนวคิดเดิมมาเปลี่ยนเป็น



จนสุดท้ายจะได้



2. ผลลัพธ์ที่นักศึกษาจะได้

เมื่อทำใบงานเสร็จ นักศึกษาจะสามารถ

1. สร้าง React Project ด้วย Vite ได้
2. อธิบาย JSX ได้
3. สร้าง React Component ได้
4. ส่งข้อมูลด้วย Props ได้
5. จัดการข้อมูลด้วย useState() ได้
6. จัดการ Event เช่น onClick() และ onChange() ได้
7. สร้าง Interactive Product List ได้
8. สร้าง Product Filter ได้
9. ใช้ useEffect() ได้
10. ดึงข้อมูลจาก REST API ได้
11. แสดง Loading / Error / Empty State ได้
12. ใช้ Tailwind CSS ร่วมกับ React ได้
13. เปลี่ยน MiniShop จาก Static UI เป็น Interactive Web Application ได้

3. เนื้อหาการเรียนรู้

Part	หัวข้อ
Part 1	เตรียม Environment + Review Week 3
Part 2	สร้าง React Project ด้วย Vite
Part 3	JSX
Part 4	Component
Part 5	Props
Part 6	State ด้วย useState
Part 7	Event Handling
Part 8	สร้าง MiniShop React
Part 9	Product Filter + Interactive Form
Part 10	useEffect + Data Fetching
Part 11	Loading / Error / Empty State
Part 12	Mini Project + Assignment

PART 1

เตรียม Environment และทบทวน Week 3

1.1 ตรวจสอบ Node.js

เปิด VS Code เลือก Terminal -> New Terminal พิมพ์

node -v

ควรได้ประมาณ **v20.x.x** หรือสูงกว่า จากนั้น

npm -v

ถ้าแสดงเลข version แสดงว่าใช้งาน npm ได้

Vite รุ่นปัจจุบันต้องการ Node.js รุ่นที่รองรับตามเอกสารของ Vite โดยปัจจุบันอยู่ที่ Node.js 20.19+ หรือ 22.12+ ดังนั้นถ้าเครื่องนักศึกษารุ่นเก่า ควรตรวจสอบก่อนเริ่ม

1.2 ทบทวน MiniShop จาก Week 3

ใน Week 3 นักศึกษาน่าจะมีโครงสร้างประมาณ

```
minishop-dashboard
|
├─ index.html
├─ package.json
├─ vite.config.js
└─ src
   └─ main.js
      └─ style.css
```

ใน Week 3 จะใช้

document.querySelector('#app').innerHTML = `

...

`

แต่ใน React เราจะเปลี่ยนวิธีคิดเป็น

HTML UI



JSX



Component



React

PART 2

สร้าง React Project ด้วย Vite

2.1 ทำไมต้องสร้าง Project ใหม่

ใน Week 3 เราสร้าง MiniShop Vanilla JS ใน Week 6 เราจะสร้าง

MiniShop React

เพื่อให้ให้นักศึกษาเปรียบเทียบได้ว่า

Week 3

HTML + JS



`document.querySelector()`



`innerHTML`

กับ

Week 6

React



Component



JSX

2.2 เปิด Terminal

ไปที่ Folder ที่เก็บ Project เช่น

C:\web_101\minishop

ใช้คำสั่ง

cd C:\web_101\minishop

2.3 สร้าง React Project

พิมพ์

npm create vite@latest minishop-react

จากนั้นเลือก

Framework:

React

จากนั้น

Variant:

JavaScript

จะได้

minishop-react

2.4 เข้า Project

cd minishop-react

2.5 ติดตั้ง Package

npm install

2.6 เปิด Project ด้วย VS Code

โครงสร้างจะประมาณนี้

```
minishop-react
|
├─ node_modules
├─ public
├─ src
|   ├─ assets
|   ├─ App.jsx
|   ├─ index.css
|   └─ main.jsx
|
├─ index.html
├─ package.json
└─ vite.config.js
```

2.7 Run Project

พิมพ์

npm run dev

จะได้ประมาณ

Local: http://localhost:5173/

เปิด Browser

http://localhost:5173

ถ้าเห็นหน้า React แสดงว่า

✓ React Project ทำงานแล้ว

PART 3

3.1 JSX คืออะไร

JSX ทำให้เราเขียน UI คล้าย HTML อยู่ภายใน JavaScript

ตัวอย่าง

```
function App() {  
  
  return (  
  
    <h1>Hello MiniShop</h1>  
  
  )  
  
}
```

สังเกตว่า

```
<h1>Hello MiniShop</h1>
```

ดูเหมือน HTML

แต่จริง ๆ แล้วอยู่ใน JavaScript

นี้เรียกว่า

JSX

3.2 เปิดไฟล์

เปิด

src/App.jsx

ลบโค้ดเดิมทั้งหมด

แล้วใส่

```
function App() {  
  
  return (  
  
    <h1>Hello MiniShop</h1>  
  
  )  
  
}
```

export default App

กด

Ctrl + S

กลับไป Browser

ควรเห็น

Hello MiniShop

3.3 JSX กับ HTML ต่างกันอย่างไร?

ตัวอย่าง HTML เดิม

```
<div class="box">
```

Hello

```
</div>
```

ใน React JSX ต้องเขียน

```
<div className="box">
```

Hello

```
</div>
```

จำให้ดี

HTML ใช้ class แต่ JSX จะใช้ className

เพราะ class เป็น keyword ที่ React/JavaScript ต้องจัดการต่างจาก HTML

3.4 ทดลอง Tailwind

เปิด

src/index.css

ใน Tailwind v4 ใช้

```
@import "tailwindcss";
```

การติดตั้ง Tailwind v4 กับ Vite ใช้ tailwindcss และ @tailwindcss/vite พร้อมนำเข้า Tailwind ด้วย @import "tailwindcss";

ถ้า Project นี้ยังไม่ได้ติดตั้ง Tailwind ให้เปิด Terminal ที่ root project แล้วใช้

```
npm install tailwindcss @tailwindcss/vite
```

จากนั้นเปิด

vite.config.js

ใส่

```
import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'
```

```
import tailwindcss from '@tailwindcss/vite'
```

```
export default defineConfig({
```

```
  plugins: [
```

```
    react(),  
    tailwindcss()  
  ]  
})
```

3.5 ทดสอบ Tailwind + JSX

กลับไป

src/App.jsx

เปลี่ยนเป็น

```
function App() {  
  return (  
    <div className="min-h-screen bg-gray-100 p-10">  
      <h1 className="text-4xl font-bold text-blue-600">  
        MiniShop  
      </h1>  
      <p className="mt-4 text-gray-600">  
        My First React Application  
      </p>  
      <button className="mt-6 bg-blue-600 text-white px-6 py-3 rounded-lg">  
        Shop Now  
      </button>
```

```
</div>
```

```
)
```

```
}
```

```
export default App
```

ผลลัพธ์

MiniShop

My First React Application

[Shop Now]

จุดที่ต้องเข้าใจ

ก่อนหน้านี้เราใช้

```
<div class="bg-blue-500 text-white p-6">
```

ตอนนี้ React จะเป็น

```
<div className="bg-blue-500 text-white p-6">
```

Tailwind ยังเหมือนเดิม

สิ่งที่เปลี่ยนคือ

HTML → JSX

class → className

PART 4

React Component

ตอนนี้เราจะเริ่มเปลี่ยน MiniShop ให้เป็นระบบ Component

4.1 Component คืออะไร

แทนที่จะเขียนทุกอย่างใน App.jsx

เราจะแยกเป็นส่วน ๆ



แนวคิดคือ หนึ่ง Component รับผิดชอบ UI หรือหน้าที่หนึ่งอย่าง

4.2 สร้าง Folder components

ใน

src

สร้าง Folder

components

เป็น



4.3 สร้าง Header.jsx

สร้างไฟล์

src/components/Header.jsx

ใส่

```
function Header() {  
  
  return (  
  
    <header className="bg-white shadow p-6">  
  
      <h1 className="text-2xl font-bold text-blue-600">  
  
        MiniShop  
  
      </h1>  
  
    </header>  
  
  )  
  
}  
  
export default Header
```

4.4 เรียก Component จาก App.jsx

เปิด

src/App.jsx

ใส่

```
import Header from './components/Header'

function App() {

  return (

    <div className="min-h-screen bg-gray-100">

      <Header />

      <main className="p-10">

        <h2 className="text-3xl font-bold">

          Dashboard

        </h2>

      </main>

    </div>

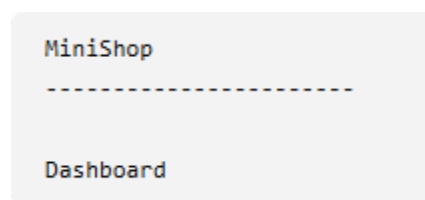
  )

}

export default App
```

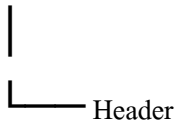
4.5 ผลลัพธ์

Browser



สิ่งที่เกิดขึ้นคือ

App



นี่คือแนวคิด Component

4.6 สร้าง ProductCard

สร้าง

src/components/ProductCard.jsx

ใส่

```
function ProductCard() {

  return (

    <div className="bg-white p-6 rounded-xl shadow">

      <div className="text-5xl text-center">

      </div>

      <h3 className="mt-4 text-xl font-bold">

        Headphones

      </h3>
```

```
<p className="mt-2 text-gray-500">
```

Wireless Headphones

```
</p>
```

```
<p className="mt-4 text-xl font-bold text-blue-600">
```

฿1,290

```
</p>
```

```
<button className="mt-4 w-full bg-blue-600 text-white py-2 rounded-lg">
```

Add to Cart

```
</button>
```

```
</div>
```

```
)
```

```
}
```

```
export default ProductCard
```

4.7 เรียก ProductCard

แก้ไข

src/App.jsx

เป็น

```
import Header from './components/Header'

import ProductCard from './components/ProductCard'

function App() {

  return (

    <div className="min-h-screen bg-gray-100">

      <Header />

      <main className="max-w-7xl mx-auto p-10">

        <h2 className="text-3xl font-bold mb-6">

          Products

        </h2>

        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">

          <ProductCard />

          <ProductCard />

          <ProductCard />

        </div>

      </main>

    </div>

  )

}

export default App
```

4.8 เกิดอะไรขึ้น?

เราตั้ง

```
<ProductCard />
```

3 ครั้ง

จึงได้ Card 3 ใบ

ProductCard

ProductCard

ProductCard

แต่มีปัญหาคือ ทุก Card มีข้อมูลเหมือนกัน ดังนั้นเราต้องเรียนรู้ **Props**

PART 5

5.1 Props คืออะไร

Props คือ ข้อมูลที่ส่งจาก Parent Component ไปยัง Child Component

ภาพจำ



เช่น

ProductCard

ชื่อ = Laptop

ราคา = 12900

รูป = 

5.2 แก้ ProductCard.jsx

เปิด

src/components/ProductCard.jsx

เปลี่ยนเป็น

```
function ProductCard({ name, price, icon }) {  
  
  return (  
  
    <div className="bg-white p-6 rounded-xl shadow">  
  
      <div className="text-5xl text-center">  
  
        {icon}  
  
      </div>  
  
      <h3 className="mt-4 text-xl font-bold">  
  
        {name}  
  
      </h3>  
  
      <p className="mt-4 text-xl font-bold text-blue-600">  
  
        ₪ {price}  
  
      </p>  
  
      <button className="mt-4 w-full bg-blue-600 text-white py-2 rounded-lg">  
  
        Add to Cart  
  
      </button>  
    </div>  
  )  
}
```

```
        </div>

    )

}

export default ProductCard
```

5.3 ส่ง Props จาก App.jsx

แก้ไข

src/App.jsx

ส่วน ProductCard เป็น

```
<ProductCard

    name="Laptop"

    price="12900"

    icon="💻"
```

```
/>
```

```
<ProductCard

    name="Headphones"

    price="1290"

    icon="🎧"
```

```
/>
```

```
<ProductCard

    name="Backpack"
```

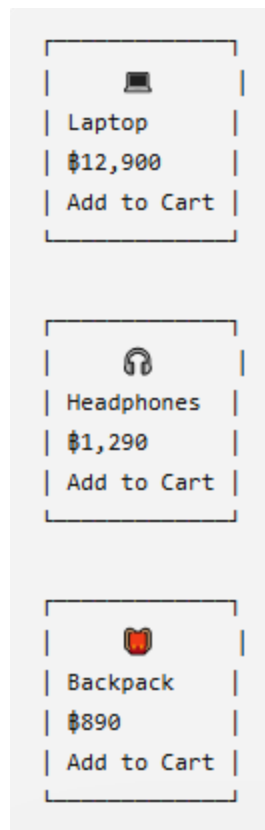
```
price="890"
```

```
icon="🎒"
```

```
/>
```

5.4 ผลลัพธ์

จะได้



สรุป Props

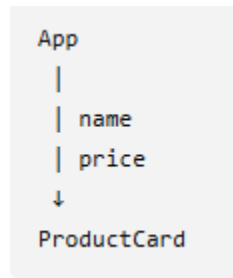
```
<ProductCard
```

```
name="Laptop"
```

```
price="12900"
```

```
/>
```

ข้อมูลเดินทางแบบ



ภายใน Component

```
function ProductCard({ name, price }) {
```

PART 6

State ด้วย useState()

นี่คือจุดที่ MiniShop เริ่มกลายเป็น **Interactive Application**

6.1 State คืออะไร

สมมติเรามีจำนวนสินค้าในตะกร้า ตอนเริ่ม

Cart: 0

กด Add to Cart

Cart: 1

กดอีกครั้ง

Cart: 2

ข้อมูลที่เปลี่ยนแปลงระหว่างการใช้งานเรียกว่า

State

6.2 import useState

เปิด

src/App.jsx

ด้านบนเพิ่ม

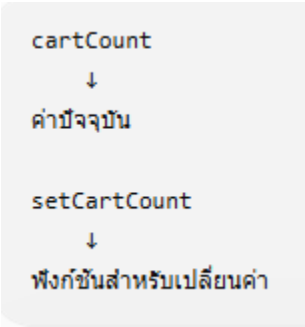
```
import { useState } from 'react'
```

6.3 สร้าง State

ภายใน App()

```
const [cartCount, setCartCount] = useState(0)
```

หมายความว่า



```
cartCount
  ↓
ค่าปัจจุบัน

setCartCount
  ↓
ฟังก์ชันสำหรับเปลี่ยนค่า
```

เริ่มต้น

0

6.4 แสดง Cart

เพิ่ม

<p>

Cart: {cartCount}

</p>

6.5 ทดลองเปลี่ยน State

เพิ่ม

```
<button  
  
  onClick={() => setCartCount(cartCount + 1)}  
  
  className="bg-blue-600 text-white px-4 py-2 rounded-lg"  
  
>  
  
  Add  
  
</button>
```

6.6 โค้ดเต็ม

```
import { useState } from 'react'  
  
function App() {  
  
  const [cartCount, setCartCount] = useState(0)  
  
  return (  
  
    <div className="p-10">  
  
      <h1 className="text-3xl font-bold">  
  
        MiniShop  
  
      </h1>  
  
      <p className="mt-4">  
  
        Cart: {cartCount}  
  
      </p>
```

```

    <button
      onClick={() => setCartCount(cartCount + 1)}
      className="mt-4 bg-blue-600 text-white px-4 py-2 rounded-lg"
    >
      Add to Cart
    </button>
  </div>
)
}

export default App

```

6.7 ทดลอง

กด

Add to Cart

ผลลัพธ์

```

Cart: 0
↓
Cart: 1
↓
Cart: 2
↓
Cart: 3

```

นี่คือ React State

เปรียบเทียบกับ JavaScript เดิม

JavaScript ปกติอาจทำ

```
document.querySelector('#cart').innerText = cartCount
```

แต่ React ทำ

```
setCartCount(cartCount + 1)
```

React จะจัดการการ Render UI ให้ใหม่ตาม State

PART 7

Event Handling

ใน React เราสามารถจัดการ Event เช่น

onClick

onChange

onSubmit

7.1 onClick

ตัวอย่าง

```
<button onClick={handleClick}>
```

Click Me

```
</button>
```

สร้าง Function

```
function handleClick() {
```

```
  alert('Hello MiniShop')
```

```
}
```

7.2 ทดลองใน App.jsx

```
function handleClick() {  
  
    alert('Product added!')  
  
}
```

แล้ว

```
<button  
  
    onClick={handleClick}  
  
    className="bg-blue-600 text-white px-4 py-2 rounded"  
  
>
```

Add to Cart

```
</button>
```

เมื่อกดปุ่ม

Add to Cart

จะเกิด

Product added!

7.3 onChange

สร้างช่องค้นหา

```
<input  
  
    type="text"
```

```
placeholder="Search product..."
```

```
className="border p-3 rounded-lg"
```

```
/>
```

ตอนผู้ใช้พิมพ์ เราต้องการอ่านค่าที่พิมพ์

```
onChange={(e) => console.log(e.target.value)}
```

รวมกัน

```
<input
```

```
type="text"
```

```
placeholder="Search product..."
```

```
onChange={(e) => console.log(e.target.value)}
```

```
className="border p-3 rounded-lg"
```

```
/>
```

เปิด Developer Tools

F12 -> Console

ทดลองพิมพ์

Laptop

จะเห็น

```
L
La
Lap
Lapt
Lapto
Laptop
```

PART 8

MiniShop React

ตอนนี้เราจะรวมความรู้ทั้งหมด

JSX

Component

Props

State

Event

Tailwind

8.1 โครงสร้าง Project

ให้สร้าง

```
src
|
├── components
|   ├── Header.jsx
|   ├── ProductCard.jsx
|   └── ProductList.jsx
|
├── App.jsx
├── index.css
└── main.jsx
```

8.2 สร้าง Product Data

เปิด

src/App.jsx

เพิ่ม

```
const products = [  
  
  {  
  
    id: 1,  
  
    name: 'Laptop',  
  
    price: 12900,  
  
    icon: '💻',  
  
    category: 'Computer'  
  },  
  
  {  
  
    id: 2,  
  
    name: 'Headphones',  
  
    price: 1290,  
  
    icon: '🎧',  
  
    category: 'Audio'  
  },  
  
  {  
  
    id: 3,  
  
    name: 'Backpack',  
  
    price: 890,
```

```

      icon: '👜',

      category: 'Fashion'

    },

    {

      id: 4,

      name: 'Smart Watch',

      price: 2990,

      icon: '🕒',

      category: 'Gadget'

    }

  ]

```

8.3 ทำไมต้องใช้ Array?

เพราะเรามีสินค้า 4 รายการ

แทนที่จะเขียน

```
<ProductCard />
```

```
<ProductCard />
```

```
<ProductCard />
```

```
<ProductCard />
```

เราจะใช้

```
products.map(...)
```

8.4 สร้าง ProductList

เปิด

src/components/ProductList.jsx

ใส่

```
import ProductCard from './ProductCard'
```

```
function ProductList({ products }) {
```

```
  return (
```

```
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
```

```
      {products.map((product) => (
```

```
        <ProductCard
```

```
          key={product.id}
```

```
          name={product.name}
```

```
          price={product.price}
```

```
          icon={product.icon}
```

```
        />
```

```
      )))
```

```
    </div>
```

```
  )
```

```
}
```

```
export default ProductList
```

8.5 App ส่ง products ไป ProductList

```
import ProductList from './components/ProductList'

function App() {

  return (

    <div>

      <h1 className="text-3xl font-bold">

        Products

      </h1>

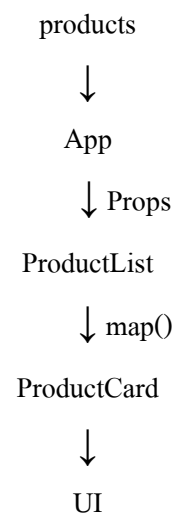
      <ProductList products={products} />

    </div>

  )

}
```

Flow ที่นักศึกษาต้องเข้าใจ ซึ่งเป็นหัวใจสำคัญของ React



PART 9

Product Filter + Interactive Form

ตอนนี้เราจะทำให้ MiniShop ใช้งานจริงมากขึ้น ต่อไปจะให้ผู้ใช้งานสามารถค้นหาสินค้าได้

9.1 สร้าง Search State

ใน

src/App.jsx

เพิ่ม

```
import { useState } from 'react'
```

แล้วใน App()

```
const [search, setSearch] = useState("")
```

9.2 สร้าง Search Box

```
<input
```

```
  type="text"
```

```
  value={search}
```

```
  onChange={(e) => setSearch(e.target.value)}
```

```
  placeholder="Search product..."
```

```
  className="w-full border p-3 rounded-lg"
```

ตรงนี้เป็นตัวอย่างของ

Two-way Binding แบบที่เราจะทำได้ใน React

9.3 Filter ข้อมูล

เพิ่ม

```
const filteredProducts = products.filter((product) =>  
  product.name.toLowerCase().includes(search.toLowerCase()  
)
```

9.4 ส่งข้อมูลที่ Filter แล้ว

เปลี่ยนจาก

```
<ProductList products={products} />
```

เป็น

```
<ProductList products={filteredProducts} />
```

9.5 ทดลอง

ค้นหา

Lap

ควรเห็น

Laptop

ค้นหา

Head

ควรเห็น

Headphones

ถ้าค้นหา

XYZ

ไม่ควรจะมีสินค้า

นี่คือ **Empty State**

PART 10

useEffect() และ Data Fetching

ตอนนี้เราจะเปลี่ยนจากข้อมูลเขียนอยู่ใน Code เป็นข้อมูลจาก API

10.1 useEffect คืออะไร

useEffect() ใช้สำหรับทำงานบางอย่างหลังจาก Component ถูก Render เช่น เรียก API อ่านข้อมูล

การตั้งค่า การสมัคร Event

รูปแบบพื้นฐาน

```
useEffect(() => {
```

```
// code
```

```
}, [])
```

10.2 Import

เปิด

src/App.jsx

เปลี่ยน

```
import { useState } from 'react'
```

เป็น

```
import { useState, useEffect } from 'react'
```

10.3 สร้าง State สำหรับ API

```
const [products, setProducts] = useState([])
```

```
const [loading, setLoading] = useState(true)
```

```
const [error, setError] = useState("")
```

เรามี State 3 ตัว

products

loading

error

10.4 เรียก API

ตัวอย่างใช้ Fake Store API

<https://fakestoreapi.com/products>

ใน useEffect

```

useEffect(() => {

  fetch('https://fakestoreapi.com/products')

    .then(response => response.json())

    .then(data => {

      setProducts(data)

      setLoading(false)

    })

    .catch(error => {

      setError('ไม่สามารถโหลดข้อมูลได้')

      setLoading(false)

    })

}, [])

```

10.5 ทำไมต้องมี []

```

useEffect(() => {

  ...

}, [])

```

หมายถึงให้ Effect ทำงานเมื่อ Component เริ่มทำงาน

โดยทั่วไปสำหรับกรณีโหลดข้อมูลครั้งแรก เราจึงใช้

[]

PART 11

Loading / Error / Empty State

11.1 Loading State

ก่อนแสดงสินค้า

```
if (loading) {  
  
  return (  
  
    <div className="p-10 text-center">  
  
      Loading products...  
  
    </div>  
  
  )  
}
```

ผู้ใช้งานเห็น

Loading products...

ระหว่างรอ API

11.2 Error State

เพิ่ม

```
if (error) {  
  
  return (  
  
    <div className="p-10 text-center text-red-600">
```

```

        {error}

      </div>

    )
  }

```

11.3 Empty State

หลังจาก Filter

```

{filteredProducts.length === 0 ? (

  <div className="text-center p-10">

    ไม่พบสินค้าที่ค้นหา

  </div>

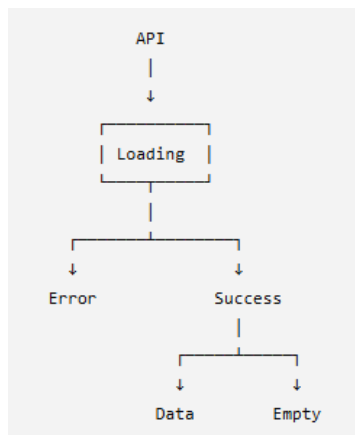
) : (

  <ProductList products={filteredProducts} />

)}

```

11.4 ตอนนี Application มี 3 สถานะ



PART 12

MiniShop Final Integration

ตอนนี้ให้รวมทุกสิ่งเข้าด้วยกัน

12.1 Application Structure

สุดท้าย Project ควรมีลักษณะ

```
minishop-react
|
├─ public
|
├─ src
|   |
|   ├─ components
|   |   ├─ Header.jsx
|   |   ├─ ProductCard.jsx
|   |   └─ ProductList.jsx
|   |
|   ├─ App.jsx
|   ├─ index.css
|   └─ main.jsx
|
├─ index.html
├─ package.json
└─ vite.config.js
```

12.2 หน้าจอที่ต้องได้

Header



Search

Products

			
Laptop	Headphone	Backpack	SmartWatch
฿12,900	฿1,290	฿890	฿2,990
[Add]	[Add]	[Add]	[Add]

Cart

เมื่อกด

Add to Cart

ตัวเลขด้านบนต้องเพิ่มค่า



เป็น



และ



PART 13

Assignment

ให้นักศึกษาพัฒนา MiniShop ต่อจากตัวอย่าง โดยต้องมียังน้อย

Requirement 1 - Component

สร้างอย่างน้อย

Header

ProductCard

ProductList

Profile

Requirement 2 - Props

ProductCard ต้องรับ

name

price

image/icon

category

ผ่าน Props

Requirement 3 - State

ต้องมี State อย่างน้อย

cartCount

search

Requirement 4 - Event

ต้องมี

onClick

onChange

Requirement 5 - Product Filter

ผู้ใช้งานสามารถ

ค้นหาสินค้า

ได้

เช่น

Search: laptop

แสดงเฉพาะ

Laptop

Requirement 6 - API

ให้ดึงข้อมูลสินค้าผ่าน API จาก Fake Store API

Requirement 7 — Loading

ต้องมี

Loading...

Requirement 8 - Error

หาก API ใช้งานไม่ได้ ต้องแสดง

ไม่สามารถโหลดข้อมูลได้

Requirement 9 - Empty State

หากค้นหาแล้วไม่พบสินค้า

ไม่พบสินค้าที่ค้นหา

Part 14 Bonus Point

นักศึกษาที่ทำงานหลักเสร็จแล้ว ให้เพิ่มความสามารถดังต่อไปนี้

Challenge 1

เพิ่ม Category Filter

All

Computer

Audio

Fashion

Gadget

Challenge 2

เพิ่มปุ่ม

Sort Price Low → High

Sort Price High → Low

Challenge 3

เพิ่ม Product Detail

เมื่อคลิก

View Detail

แสดง

ชื่อสินค้า

ราคา

รายละเอียด

Rating

Challenge 4

เพิ่ม Cart

เมื่อ Add Product

Cart: 1

เมื่อ Add อีกครั้ง

Cart: 2

Part 15 ปัญหาที่อาจเกิดขึ้น

ปัญหา 1

ขึ้น

Failed to resolve import

ให้ตรวจว่าไฟล์มีจริงหรือไม่ เช่น

src/components/ProductCard.jsx

และ import ต้องตรง

```
import ProductCard from './components/ProductCard'
```

ปัญหา 2

ขึ้น

ProductCard is not defined

ตรวจว่ามี

```
import ProductCard from './components/ProductCard'
```

หรือไม่

ปัญหา 3

หน้าเว็บขาว

ใน Browser ให้เปิด Inspect -> Console

อ่าน Error สีแดง

จากนั้นตรวจสอบ

App.jsx

ก่อน

ปัญหา 4

Tailwind ไม่ทำงาน

ตรวจสอบ

src/index.css

ว่ามี

```
@import "tailwindcss";
```

หรือไม่

และตรวจสอบ

vite.config.js

ว่ามี

```
import tailwindcss from '@tailwindcss/vite'
```

และ

```
tailwindcss()
```

หรือไม่

ปัญหา 5

API ไม่แสดงข้อมูล

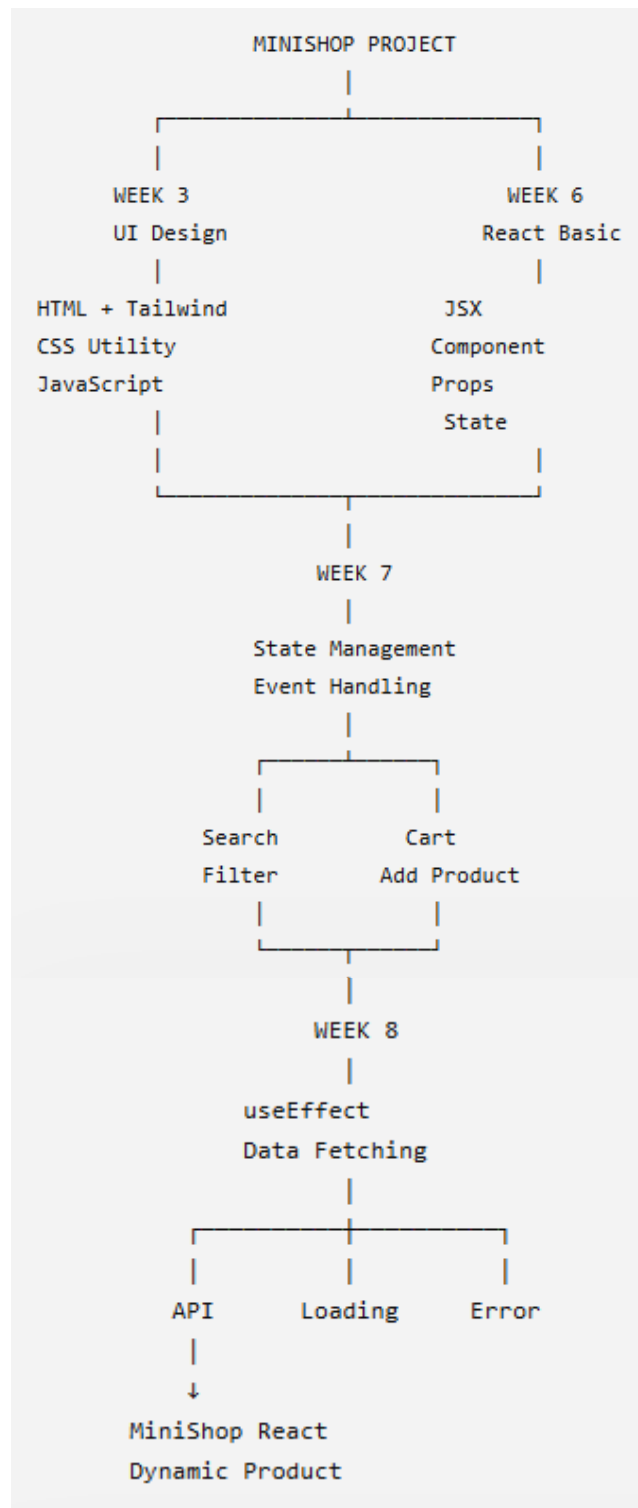
ในหน้า Brower ให้เปิด Inspect -> Console

และ

ในหน้า Brower ให้เปิด Inspect -> Network

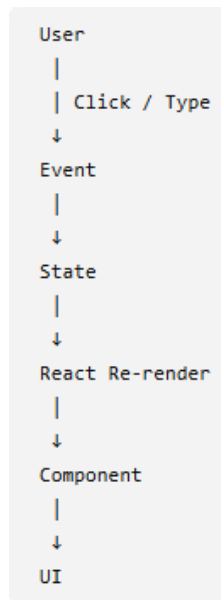
ตรวจ Request ที่เรียก API

16. ภาพรวมการเรียนรู้ทั้ง 4 สัปดาห์



18. Learning Outcome ที่สอดคล้องกับใบงาน

เมื่อจบชุดใบงานนี้ นักศึกษาควรอธิบาย Flow นี้ได้ด้วยตนเอง



และเมื่อเข้าสู่ Week 8

